

SmartHVAC Studio

Project Architecture & System Documentation

ME420 — Mechanical Engineering Research Project

Department of Mechanical Engineering

University of Moratuwa

April 29, 2026

Contents

1	Introduction	2
2	Repository Structure	2
3	Five-Layer Architecture	2
3.1	Layer 1 — Web Frontend	4
3.1.1	User Interface Pages	4
3.1.2	Client Logic (<code>script.js</code>)	4
3.2	Layer 2 — Firebase Cloud Services	4
3.3	Layer 3 — Colab Backend Worker	4
3.4	Layer 4 — Agentic AI Pipeline	5
3.4.1	Two-Pass Agentic RAG Workflow	5
3.4.2	IDF Assembly	6
3.5	Layer 5 — EnergyPlus Simulation Engine	6
4	End-to-End Data Flow	6
5	Key Design Decisions	7
6	Conclusion	8

1 Introduction

SmartHVAC Studio is a cloud-based research platform that enables users to generate and simulate building energy models through natural language descriptions. A user describes a building in plain English (e.g., “Create a 15×15 building with brick walls”), and the system autonomously produces a valid EnergyPlus Input Data File (IDF), executes the thermal simulation on a GPU-equipped Google Colab runtime, and returns time-series result plots to a web dashboard.

The platform is designed around a **5-Layer Architecture** in which the Web Frontend, Firebase Cloud Services, a Colab Backend Worker, an Agentic AI Pipeline, and the EnergyPlus Simulation Engine each operate as independent, loosely coupled layers. This document explains every layer, every source file, and the data flow that connects them.

2 Repository Structure

The two primary directories are `web/` (Layer 1) and `colab/` (Layers 3–5). Their contents are summarised below.

Table 1: Key files in the `web/` directory (Layer 1).

File	Type	Purpose
<code>index.html</code>	HTML	Landing page with navigation sidebar.
<code>pages/nlp.html</code>	HTML	Simulation Setup form: text area, model selector, weather file picker.
<code>pages/results.html</code>	HTML	Job history table, result plots, View IDF and View Object Summary buttons.
<code>pages/idf_viewer.html</code>	HTML	Standalone IDF text viewer.
<code>pages/architecture.html</code>	HTML	In-app architecture diagram page.
<code>script.js</code>	JS	All client logic: job submission, polling, plot loading, IDF/summary viewers.
<code>firebaseConfig.js</code>	JS	Firebase project credentials (excluded from version control).
<code>style.css</code>	CSS	Global styles, dark theme, card system, responsive layout.
<code>design_system.css</code>	CSS	Reusable design tokens (colours, spacing, typography).

3 Five-Layer Architecture

Figure 1 provides a high-level overview of the five layers and their primary communication channels.

Table 2: Key files in the `colab/` directory (Layers 3–5).

File	Type	Purpose
<code>Run_Connected_Experiment.ipynb</code>	Notebook	Main orchestrator: mounts Drive, boots Ollama, runs the polling loop.
<code>simulation_runner.py</code>	Python	EnergyPlus execution wrapper; post-processing (plots, CSV, <code>summary.html</code>).
<code>backend/ai_generator.py</code>	Python	<code>AIPipelines</code> class: two-pass Agentic RAG, multi-provider LLM calls.
<code>backend/firebase_connector.py</code>	Python	<code>FirestoreConnector</code> class: Firestore reads/writes, Storage uploads.
<code>backend/geometry_util.py</code>	Python	<code>generate_zone_geometry()</code> : 3-D vertex maths for walls, roof, floor, and windows.
<code>backend/idf_extractor.py</code>	Python	Recursive dependency resolver: fetches Construction $\rightarrow$ Material chains from Datasets.
<code>backend/dataset_indexer.py</code>	Python	One-time script that scans <code>Datasets/</code> and writes <code>index.json</code> .
<code>backend/index.json</code>	JSON	Lookup table: 7000+ EnergyPlus objects mapped to their source <code>.idf</code> files.
<code>templates/Base.idf</code>	IDF	Parameterised EnergyPlus template with <code>{PLACEHOLDER}</code> tokens.
<code>serviceAccountKey.json</code>	JSON	Firebase Admin SDK credential (excluded from version control).
<code>secrets.json</code>	JSON	API keys for OpenAI, Gemini, and HuggingFace.

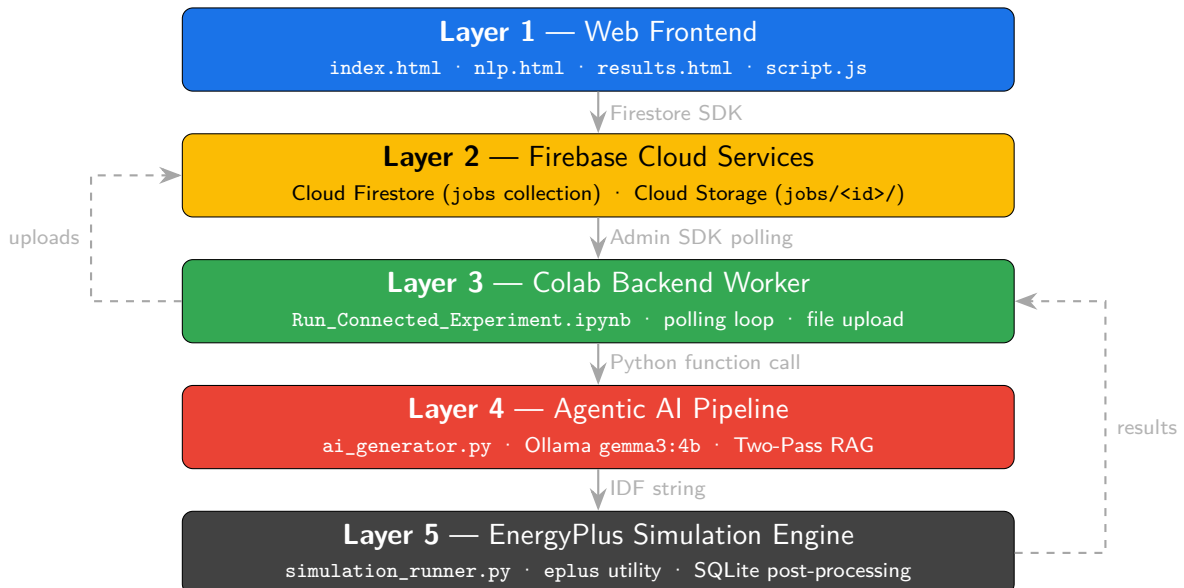


Figure 1: Five-layer architecture stack and primary data channels.

3.1 Layer 1 — Web Frontend

The frontend is a static single-page application served locally via Python's built-in HTTP server (`python -m http.server 8000`). It consists of vanilla HTML, CSS, and JavaScript—no build step or framework is required.

3.1.1 User Interface Pages

- `nlp.html`: The user types a natural-language building description, selects an AI model (`openai`, `gemini`, `huggingface`, or `ollama`), and chooses a weather file. Clicking *Submit* calls `submitDescription()` in `script.js`.
- `results.html`: Displays a table of all submitted jobs. Clicking a row calls `showJobDetails()`, which renders status badges, result plot images, and two action buttons: *View Generated IDF* and *View Object Summary*.

3.1.2 Client Logic (`script.js`)

The script initialises Firebase using the credentials exported from `firebaseConfig.js`, then exposes the following key functions:

`submitDescription()` Writes a new document to the Firestore `jobs` collection with `status: "queued"`.

`loadJobs()` Reads all documents from `jobs`, sorts by timestamp, and populates the results table.

`showJobDetails()` Fetches plot images from Firebase Storage using `storage.ref().getDownloadURL()` and sets them as `<img>` sources (images bypass CORS natively).

`viewIDF()` Opens the backend-generated `diff.html` (a side-by-side diff of Base vs. Generated IDF) in a new tab.

`viewIDFSummary()` Opens the backend-generated `summary.html` in a new tab, displaying all IDF objects in expandable accordion cards.

`testAIConnection()` Writes a test document to Firestore's `test_connectivity` collection and waits for the backend to respond.

3.2 Layer 2 — Firebase Cloud Services

Firebase acts as the stateless coordination bus between the browser and the Colab worker. Two services are used:

- **Cloud Firestore** — A NoSQL document database. The `jobs` collection stores one document per simulation request. Fields include `status` (`queued` → `running` → `done` / `error`), `nlpInputText`, `simulationConfig`, and `resultPaths`.
- **Cloud Storage** — Binary file storage. The backend uploads plot PNGs, the generated IDF, the HTML diff, and the HTML summary to the path `jobs/<jobId>/`.

No custom Cloud Functions or servers are deployed; the Colab notebook polls Firestore directly.

3.3 Layer 3 — Colab Backend Worker

The Jupyter notebook `Run_Connected_Experiment.ipynb` is executed on a Google Colab GPU runtime. Its cells perform the following boot sequence:

1. Mount Google Drive to access the repository files.

2. Copy the pre-downloaded Ollama model weights from Drive to the Colab SSD (/root/.ollama/models/) for fast memory-mapped inference.
3. Start the Ollama server with `nohup ollama serve` and wait for readiness.
4. Initialise `FirestoreConnector` and `AIPipelines`.
5. Enter an infinite polling loop that queries Firestore every 3 seconds for documents where `status == "queued"`.

When a queued job is found, the loop:

1. Sets `status` to "running".
2. Calls `ai.generate_idf_from_text()` to produce the IDF string (Layer 4).
3. Saves the IDF to `RunFiles/<jobId>_in.idf`.
4. Calls `simulation_runner.run_simulation_job()` to execute EnergyPlus (Layer 5).
5. Uploads every result file (plots, CSV, IDF, `diff.html`, `summary.html`) to Firebase Storage.
6. Sets `status` to "done" and writes `resultPaths`.

3.4 Layer 4 — Agentic AI Pipeline

The AI pipeline lives in `backend/ai_generator.py` inside the `AIPipelines` class. It supports four LLM backends: OpenAI, Google Gemini, HuggingFace Inference, and a local Ollama instance running `gemma3:4b`.

3.4.1 Two-Pass Agentic RAG Workflow

The core generation method `generate_idf_from_text()` implements a Retrieval-Augmented Generation (RAG) pattern in two passes. Figure 2 illustrates the flow.

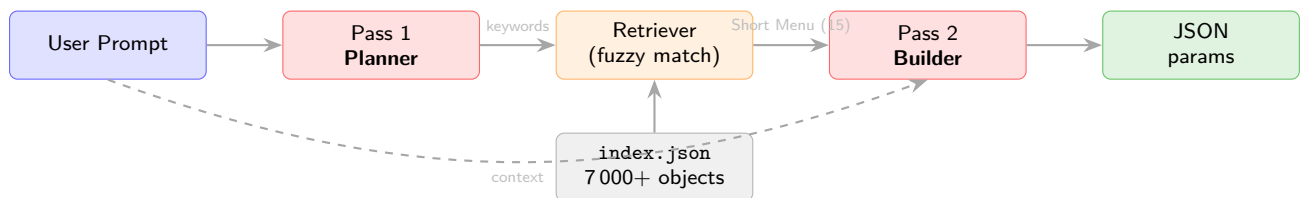


Figure 2: Two-pass Agentic RAG workflow inside `ai_generator.py`.

3.4.1.1 Pass 1 — The Planner.

The method `_generate_search_keywords()` sends the user's natural-language prompt to the LLM with instructions to return a JSON array of material and HVAC keywords. If the prompt is vague, the model is instructed to infer ASHRAE-standard defaults. The returned list typically contains 5–20 keywords such as ["wood stud", "brick", "insulation", "metal roof"].

3.4.1.2 The Retriever.

The 7000+ entries in `index.json` (built by `dataset_indexer.py`) are scored against the Planner's keywords using a simple fuzzy-matching function. The top 15 constructions form the *Short Menu*.

3.4.1.3 Pass 2 — The Builder.

A second LLM call receives the user prompt together with the Short Menu and a strict JSON schema. The schema demands 15 keys covering geometry (`length`, `width`, `height`), envelope (`wall_construction`, `roof_construction`), windows (`wwr`, `window_u_factor`, `window_shgc`), internal gains (`people_density`, `light_density`, `equipment_density`), airflow (`ventilation_ach`, `infiltration_ach`), and setpoints (`heat_set_occ`, `heat_set_unocc`, `cool_set_occ`, `cool_set_unocc`).

3.4.2 IDF Assembly

After parsing the JSON response, the assembler:

1. Calls `geometry_util.generate_zone_geometry(L, W, H, wwr)` to produce 3-D vertices for walls, roof, floor, and window sub-surfaces.
2. Calls `idf_extractor.resolve_dependencies()` for the chosen wall and roof constructions. This recursively fetches the `Construction` block and all referenced `Material` blocks from the `Datasets` folder.
3. Concatenates the parameterised `Base.idf` template, the extracted material blocks, and the geometry block.
4. Performs placeholder injection: every `{PLACEHOLDER}` token in the combined string (e.g., `{PEOPLE_DENSITY}`, `{COOL_OCC}`) is replaced with the AI-extracted numeric value.

3.5 Layer 5 — EnergyPlus Simulation Engine

`simulation_runner.py` wraps the `eplus` utility library (installed from a custom GitHub branch). Its `run_simulation_job()` function:

1. Calls `prepare_colab_eplus()` to ensure the EnergyPlus binary is available.
2. Creates an `EPlusUtil` instance and configures the model, weather, and output directory.
3. Runs the simulation (`design_day` or `annual`).
4. Opens the resulting SQLite database (`eplusout.sql`) with `EPlusSqlExplorer` and queries for output variables such as Zone Air Temperature, Cooling Energy, and Heating Energy.
5. Generates `matplotlib` plots and saves them as PNGs.
6. Parses the generated IDF text to produce `summary.html`—an HTML page with expandable accordion cards listing every EnergyPlus object and its detailed fields.

4 End-to-End Data Flow

The complete lifecycle of a single simulation job is illustrated in Figure 3 and described step-by-step below.

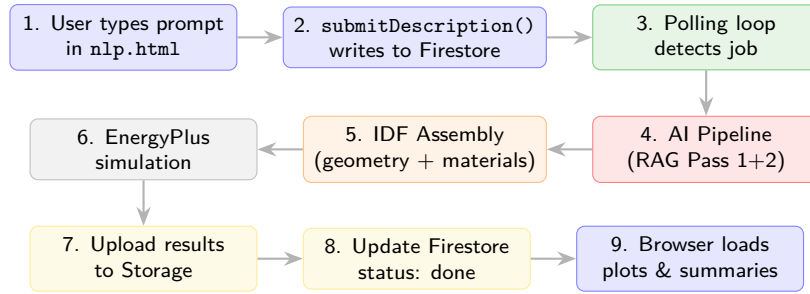


Figure 3: End-to-end lifecycle of a single simulation job.

1. **User** → **Browser**: Types a building description into `nlp.html`.
2. **Browser** → **Firestore**: `submitDescription()` writes a document with `status:"queued"` to the `jobs` collection.
3. **Colab** ← **Firestore**: The polling loop detects the queued document and marks it "running".
4. **Colab** → **Ollama**: Pass 1 extracts keywords; the Retriever filters `index.json`; Pass 2 returns a JSON parameter dictionary.
5. **Colab (Assembly)**: `geometry_util` + `idf_extractor` + `Base.idf` → complete IDF string with injected values.
6. **Colab** → **EnergyPlus**: `simulation_runner` executes the IDF against the `.epw` weather file.
7. **Colab** → **Storage**: Plots, CSV, IDF, `diff.html`, and `summary.html` are uploaded to Firebase Storage.
8. **Colab** → **Firestore**: Document updated to `status:"done"` with `resultPaths`.
9. **Browser** ← **Firestore/Storage**: `loadJobs()` refreshes the table; `showJobDetails()` loads plot images; buttons open the IDF diff and object summary.

5 Key Design Decisions

- **Serverless coordination**: Firebase eliminates the need for a dedicated backend server. The Colab notebook acts as an ephemeral worker that can be started and stopped without infrastructure changes.
- **Local LLM on Colab GPU**: Running `gemma3:4b` via Ollama on the Colab T4 GPU avoids per-token API costs and removes internet-latency from the generation loop. Model weights are persisted on Google Drive and copied to the SSD at boot to satisfy Ollama's memory-mapping requirements.
- **Agentic RAG**: The two-pass design prevents context-window overflow. Pass 1 reduces 7000+ objects to 15, enabling Pass 2 to focus on precise parameter extraction.
- **Template + Injection**: Rather than asking the LLM to write raw IDF syntax (error-prone), the system uses a human-verified `Base.idf` with placeholder tokens. The LLM only outputs structured JSON; all IDF syntax is handled deterministically.
- **CORS avoidance**: Plot images use `<img>` tags (exempt from CORS). The IDF summary is pre-rendered as `summary.html` on the backend and opened via `window.open()`, bypassing the browser's cross-origin `fetch` restrictions entirely.

6 Conclusion

SmartHVAC Studio demonstrates a feasible pipeline for translating natural-language building descriptions into validated EnergyPlus simulations. The five-layer separation of concerns allows each component—browser UI, cloud database, GPU worker, LLM intelligence, and physics engine—to evolve independently. The Agentic RAG approach, combined with deterministic template injection, achieves a practical balance between AI flexibility and simulation reliability.